

Kimi K3 深度解读

为什么这样设计 · 预训练 · 预训练 Infrastructure

Kimi K3: Open Frontier Intelligence (2026.07) 技术报告精读 · 第二讲

2.8T

总参数

104B

激活参数

1M

上下文

896 选 16

MoE 专家

2.5×

扩展效率 vs K2

本讲侧重：结构设计的动机 / 预训练与其系统实现 · 结构基础与推理服务从略

今天的重点（第二讲）

六个部分，结构『为什么』 / 预训练 / 预训练 Infra 是重心

#	章节	内容	配时
01	设计哲学与决策总表	三维信息流；一页看全『选择→原因』	5 min
02	结构设计的原因（精简）	序列 / 深度 / 宽度：为什么，而非是什么	12 min
03	预训练（扩充）	数据 / scaling law 方法论 / 1M 课程 / 稳定性	12 min
04	预训练 Infra（重点）	FlashKDA / KCP / MoonEP / 显存 / ViT	15 min
05	强化学习（简化）	三阶段 / 环境 / MOPD 一带而过	7 min
06	评测与总结	成本效率 / 案例 / 方法论启示	3 min

- 相较上一讲：结构介绍与推理服务收缩，预训练与预训练 Infra 显著扩充



01

设计哲学与决策总表

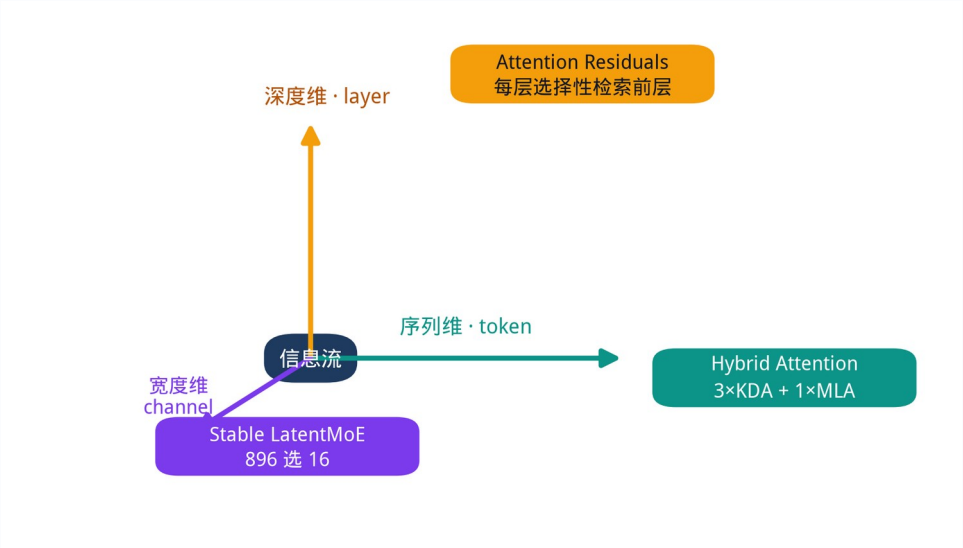
先建立统一视角：K3 的每个改动都在扩展某个维度的『信息流』

统一视角：在三个维度上扩展信息流

序列 / 深度 / 宽度，各有痛点与方案

维度	瓶颈	K3 方案
序列 token mixing	1M 下 softmax 计算 $O(T^2)$ 与 KV cache $O(T)$ 爆炸	3×KDA + 1×Gated MLA
深度 layer mixing	残差把历史层等权压成单态，深层看不清早期	Attention Residuals
宽度 channel mixing	扩专家池 → 通信 / 带宽 / 稳定性恶化	Stable LatentMoE

- 每 block：3×(KDA+MoE) + 1×(Gated MLA+MoE)，末层强制全局
- 本讲对『是什么』点到为止，重点在后面『为什么』



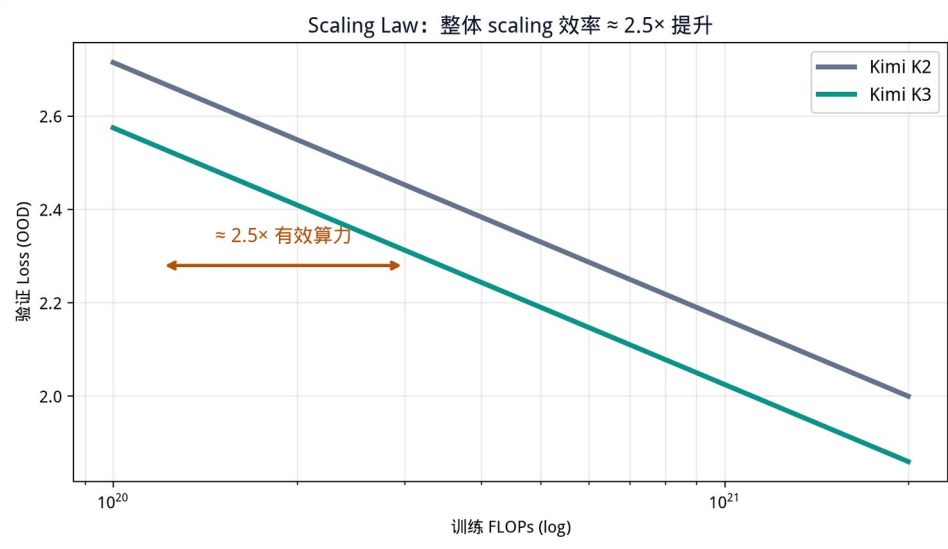
要点：KDA 跨 token · AttnRes 跨 layer · LatentMoE 跨 channel。

K2 → K3：为什么值得从头重调

加深 + 加稀疏，而非加宽

	Kimi K2	Kimi K3	Δ
层 / 总参 / 激活	61 / 1.04T / 32.6B	93 / 2.78T / 104.2B	↑ 52 / 167 / 220%
注意力	61× MLA	69× KDA + 24× MLA	混合线性
MoE	384 选 8 + 1	896 选 16 + 2 (latent 3584)	稀疏度 56
激活 / 上下文	SwiGLU / 128K	SiTU-GLU / 1M	8× ctx

- hidden 维不变 (7168)，靠加深 + 加稀疏扩张；激活仅占 3.7%
- 2.5× 是『整体』效率（架构 + 数据 + 超参共同贡献），OOD 验证集拟合



要点：改动大到改变了『最优训练 regime』→ 必须重做 scaling law 搜索。

一页看全：每个选择背后的原因

整场索引 · 痛点 → 选择 → 为什么

维度 / 模块	痛点	选择	为什么是它
序列 token mixing	1M 下 softmax 太贵	3 KDA : 1 MLA 混合	线性扛长程 + 全局保精确检索
KDA 衰减门	chunkwise 对角块有慢路径	lower-bounded decay	数值压进 BF16 → 全块走 Tensor Core
全局层位置编码	扩上下文要重调 RoPE/YaRN	NoPE	位置由 KDA 隐式携带 → 零调参外推
深度 layer mixing	残差稀释早期层	Attention Residuals	深度维照搬序列维的 attention 革命
宽度 MoE	扩专家 → 通信 / 带宽爆炸	Stable LatentMoE	latent 压缩 + 归一 / 有界 / 分位数均衡
优化器	大规模各头更新失衡	Per-Head Muon	逐头正交化均衡更新尺度

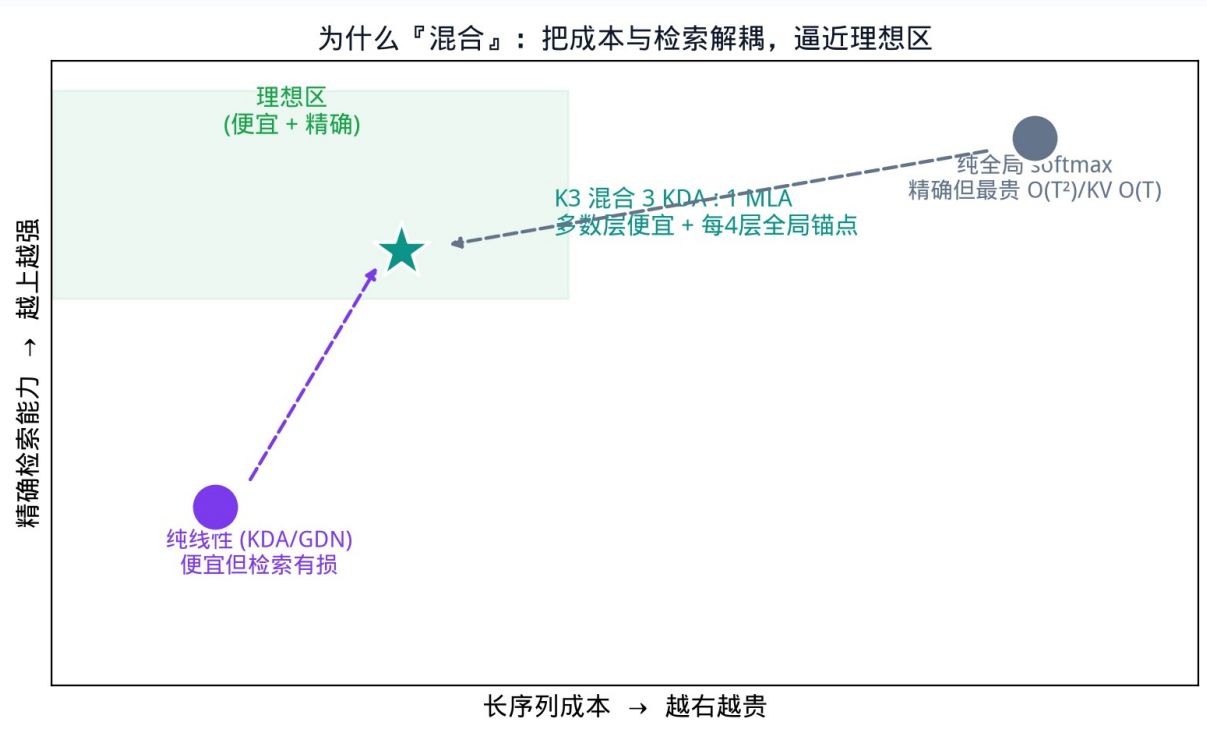
02

结构设计的原因（精简）

不讲『是什么』，只讲『为什么这样、为什么不是别的』

序列①：为什么是『混合』，而不是纯线性 / 纯全局

把成本与检索解耦



- 纯全局 softmax：精确但最贵（ $O(T^2)$ 、KV $O(T)$ ），1M 下不可行
- 纯线性：便宜但有损——固定状态是压缩，精确检索会退化
- K3：3 KDA : 1 MLA，多数层便宜、每 4 层一个全局『锚点』
 - 只有 1/4 的层贡献随 T 增长的 cache（还是低秩 MLA）
- 结论：混合把『成本』与『检索』解耦，逼近左上角理想区

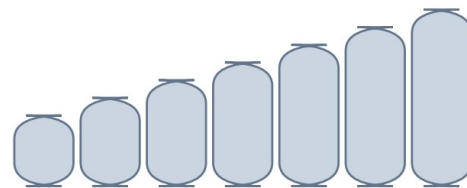
要点：不是一条道走到黑，而是让不同层各司其职。

序列②： KDA —— 唯一需要记住的一个公式

用固定状态换掉增长的 KV cache

- 把『不断增长的 KV cache』换成『固定大小状态 S 』——长序列便宜的关键
- **delta rule**：写入前先擦掉该 key 方向的旧值
 - = 对在线回归目标走一步梯度下降
- **通道级遗忘门 $\text{Diag}(\alpha)$** ：每个 key 通道独立决定遗忘多少
 - 比 GDN 的标量门更细；配 ShortConv + L2Norm + 满秩输出门

Softmax 注意力：KV cache 随长度增长



内存 $O(T) \uparrow \uparrow$

KDA：固定大小递归状态 S



内存 $O(1)$ · 易传输/复用

delta-rule 更新：先「按 key 方向擦除旧关联」，再「写入新 kv^T 」

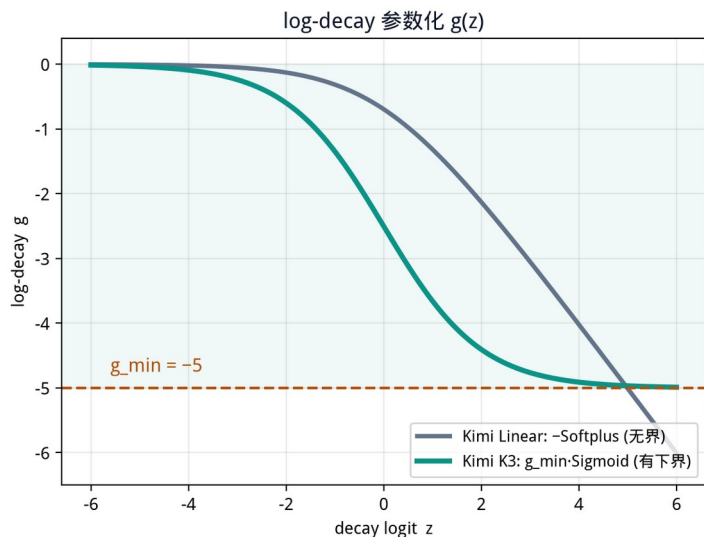
通道级遗忘门 $\text{Diag}(\alpha)$ ：每个特征维独立控制记忆衰减
(比 Gated DeltaNet / Mamba-2 的「头级标量门」更细粒度)

要点：固定大小状态——记住这点，Infra 的 KCP 与前缀缓存都建立在它之上。

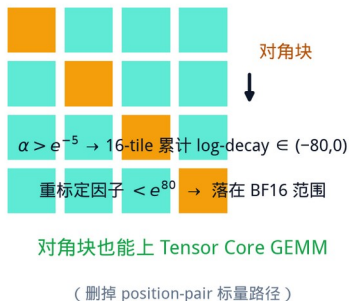
序列③：一个『为硬件让路』的小改动

改门控值域，删掉一整条 kernel 慢路径

- 问题：chunkwise 要除以累积衰减 $1/\Gamma$ ， α 连乘 $\rightarrow$ 可无界增长、溢出
- Kimi Linear 老办法：对角块逐位置对算 —— 一条慢路径，上不了 Tensor Core
- K3：把衰减门下界卡在 $g_{\min} = -5$ （左图红线）
 - 16-token 块累积衰减 $\in (-80, 0) \rightarrow$ 倒数 $< e^{80}$ 落进 BF16
- 收益：对角块也能走稠密 Tensor Core，慢路径被删除



系统后果



要点：改一个门控函数的值域，换来 kernel 一整条慢路径被删除（详见 Infra-FlashKDA）。

序列④：为什么敢用 NoPE + 给全局层加门

职责分离：KDA 扛长程 + 位置，MLA 做全局检索

- **Gated MLA**：全局注意力层，沿用 DeepSeek-V2 的 MLA（KV 压成低维 latent 再重建）
 - KV cache 小；K3 再加一个输入依赖的满秩 sigmoid 输出门（与 KDA 一致）
- **NoPE**：MLA 层完全不加位置编码
 - 为什么敢：位置信息由中间的 KDA 层通过递归 + 衰减门隐式携带
 - 红利：上下文从 8K 扩到 1M 全程零位置编码调参（无 RoPE rescale / YaRN）
- **取舍主线**：KDA 便宜地扛长程 + 位置感，MLA 周期性做无位置约束的全局检索

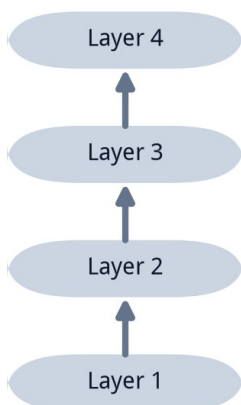
要点：NoPE 是 K3 能『直接外推到 1M』的关键之一。

深度：残差是『深度方向的 RNN』，那就换成 attention

把序列维的 attention 革命搬到深度维

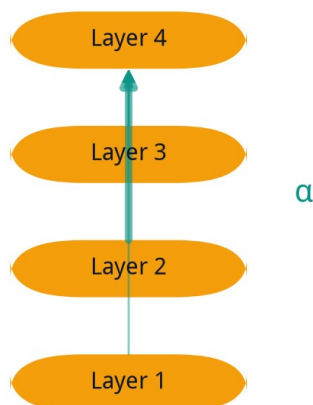
标准残差 = 深度维 RNN

固定单位权重逐层累加
早层信息被稀释、幅度 $O(L)$ 增长



Attention Residuals = 深度维 softmax

每层用可学 pseudo-query
对所有前层做 softmax 选择



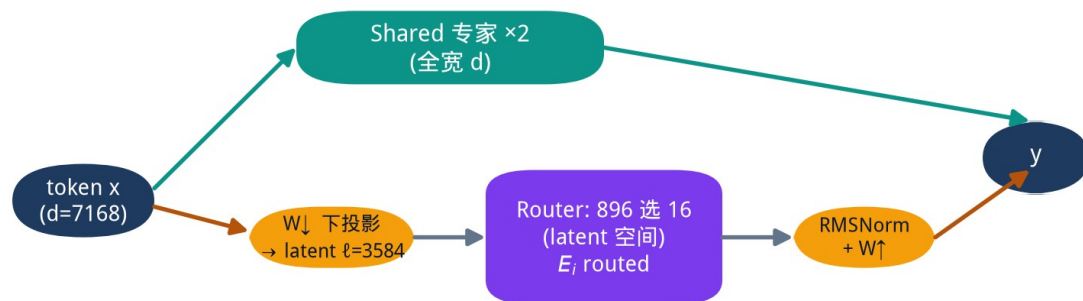
- 痛点：标准残差把之前所有层等权累加成一个状态 → 深层信息被稀释
 - PreNorm 下幅值还随深度无界增长，破坏稳定
- 类比：这正是 RNN 沿时间的瓶颈；序列维早已用 attention 解决 → 深度维照做
- 做法：每层用一个可学习 pseudo-query，对前面各层输出做 softmax 注意力
- Block 版：分块降开销 $O(Ld) \rightarrow O(Nd)$ ， $N \approx 8$ 恢复大部分收益，推理开销 $< 2\%$

要点：标准残差 = 深度维 RNN；AttnRes= 深度维 softmax。

宽度①：为什么 896 个专家还能训得起

解耦『模型全宽』与『专家宽度』

Stable LatentMoE：分离「模型宽度」与「路由专家宽度」



只在压缩 latent 空间里路由 → 896 选 16 的通信/显存才付得起（稀疏度 ≈ 56）

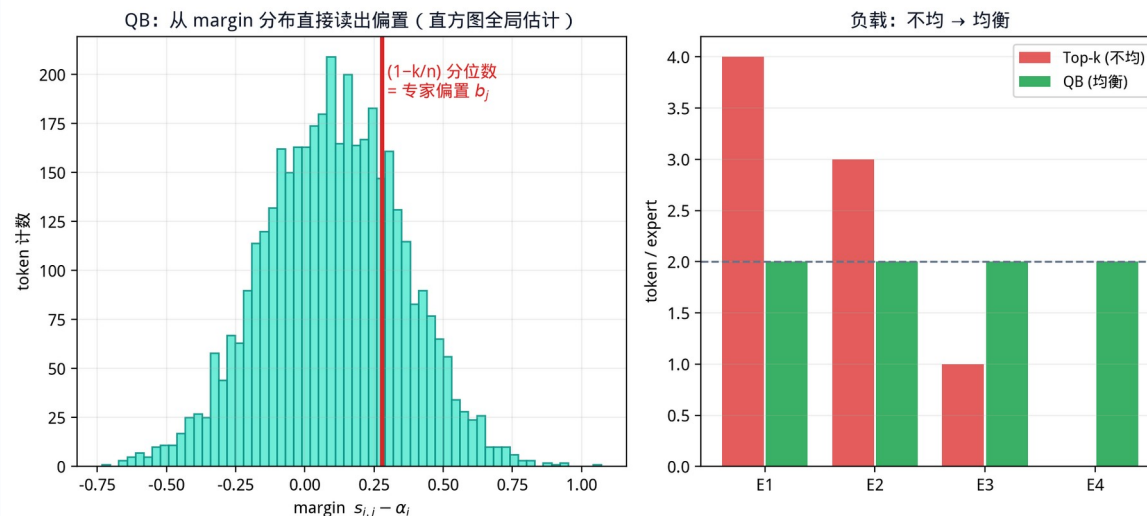
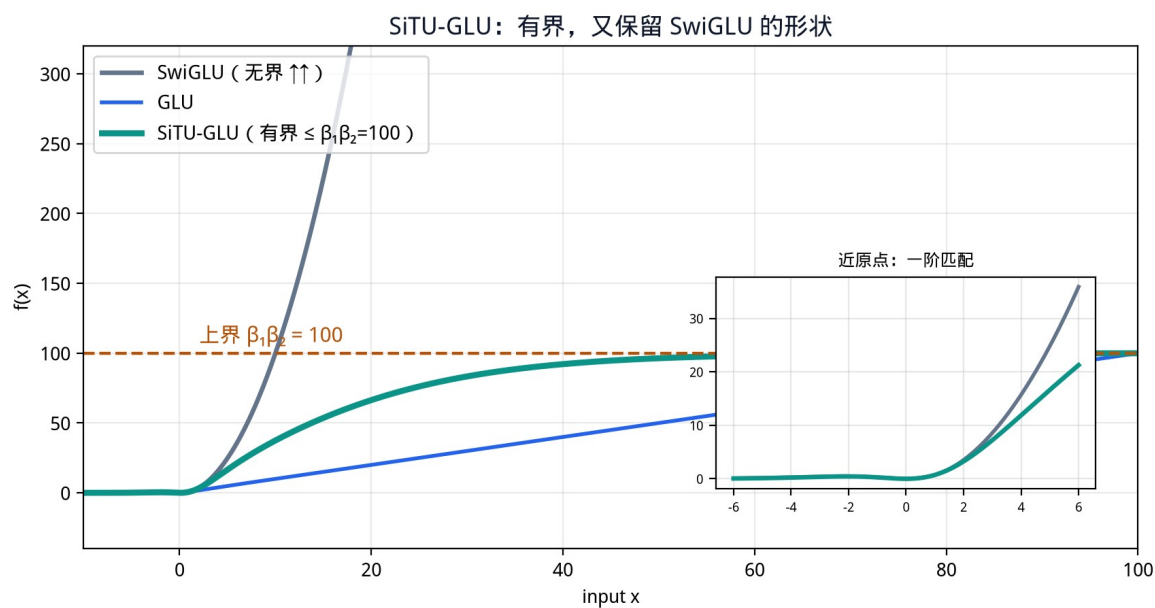
RMSNorm 治「4 连乘病态」的激活爆炸 · SiTU-GLU 有界 · QB 均衡

- 痛点：常规 MoE 里每个被选专家都吃完整 d 维 → 通信 / 权重带宽随激活数线性涨
- LatentMoE：共享专家保留全宽通路；路由专家在 $\ell = d/2 = 3584$ 的 latent 空间工作
 - dispatch/combine 通信减半、权重带宽减半；激活专家 8→16、专家 384→896
- 极端稀疏 + 2.8T 放大两个失效模式
 - ① $W \downarrow \rightarrow$ 门控 FFN $\rightarrow W \uparrow$ 近 4 连乘的病态链 → 内部激活爆炸
 - ② 近千专家的负载均衡超出传统 sign 偏置的可控范围

要点：latent 压缩让『更多、更小、更专』的专家变得可负担。

宽度②：两个稳定化设计—— SiTU-GLU 与 Quantile Balancing

一个压激活幅值，一个压负载失衡



- SiTU-GLU : 给 SwiGLU 两分支各戴光滑软帽 $\beta \cdot \tanh(x/\beta)$, 近原点 $\approx$ SwiGLU、输出 $\leq \beta_1\beta_2=100 \rightarrow$ 为 MXFP4/8 兜底
- Quantile Balancing : 把均衡看作最优指派, 专家偏置有闭式解 = 目标负载分位数; 无学习率超参、几步收敛、直方图全局估计

要点: 都服务于『极端稀疏下的稳定训练』。

横向对照：三家长上下文注意力路线

两条根本路线

	路线	代表	机制要点
线性状态混合	RNN 式固定状态 + 周期全局层	Kimi K3 / Qwen3.5	K3 通道级 $\text{Diag}(\alpha)$ + ShortConv+ 满秩门；Qwen 标量 GDN
压缩 + 稀疏检索	保 softmax，压缩 KV + top-k 选择	DeepSeek-V4	CSA 4× / HCA 128× 压缩 + 轻量 indexer 稀疏选

● K3 与 Qwen 走线性混合；DeepSeek 走稀疏 softmax —— 无绝对优劣，是两种工程审美

要点：线性状态混合 vs 压缩稀疏检索：两条根本路线。

03

预训练

数据 recipe · scaling law 方法论 · 1M 长上下文课程 · 稳定性

预训练数据：不只是『更多』，而是『更会喂』

语料构成 + 两个数据工程

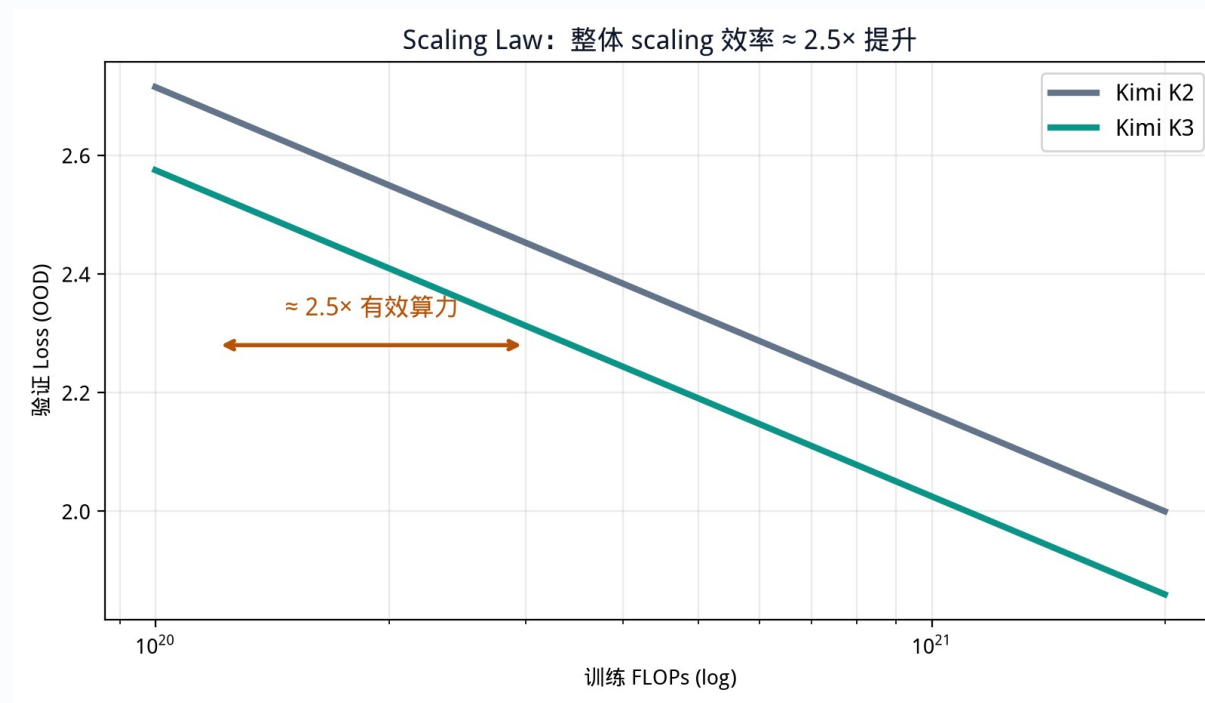
- 文本四域：Web / Code / Math / Knowledge，各经规则 + 分类器打分 + 去重，配比靠小模型消融确定
- 视觉语料：caption / 图文交错 / OCR / 感知 / 视频 / visual coding；坐标给绝对 + 归一化两种格式
- Rephrasing（承自 K2）：对知识 / 数学做风格与视角多样化改写 + 对原文保真校验
 - 让同一份知识以不同角度多次曝光，而非简单重复 → 提高每 token 知识密度
- 程序化多模态：代码片段配其渲染结果（SVG/3D/ 网页 / 游戏 / CAD）
 - → 支撑『写代码看效果』的 vision-in-the-loop 能力

要点：配比靠消融、改写要保真、程序化数据造就视觉编程能力。

方法论：一个值得抄的 scaling law 实验设计

cosine vs WSD 的公平比较

- 改动大到改变最优训练 regime → 重做 scaling law 搜索 (batch / LR / TPP / 形状)
- cosine vs WSD：两种调度的最优超参 (峰值 LR、batch) 差异很大
 - 用同一组超参比较，会不公平地偏向恰好更适配它的那个
- 正确做法：分别给两者各自调到最优，再比 → 结论才是 cosine 更优

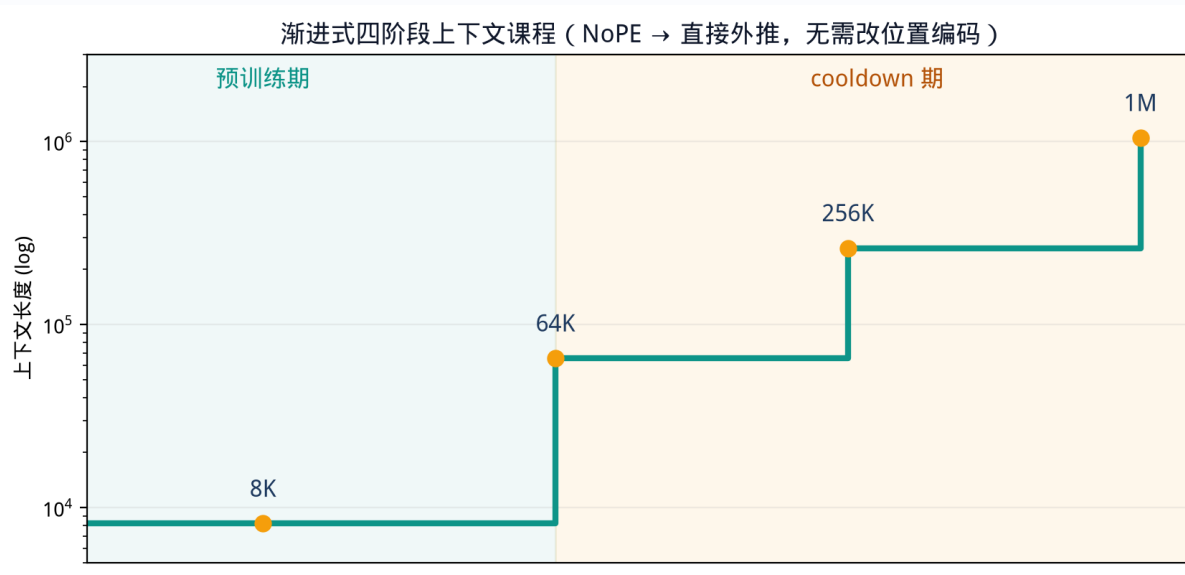


要点：比较任意训练技术前，先分别各自调到最优——几乎适用于所有 A/B 实验。

1M 上下文：四阶段课程 + 强制长依赖

课程经济学 + 长≠长程能力

- 四阶段：预训练 8K→64K，cooldown 256K→1M（把最贵的长序列计算集中在一小段预算）
- 长数据清洗：精确 + 模糊去重 / 视频帧感知哈希 / 结构校验；真长文稀缺→上采样
- 长 ≠ 长程能力：合成『必须跨全 1M 取证才能解』的拼接任务，强制 attention 在目标尺度训练
- NoPE + KDA → 全程零位置编码调参（无 RoPE rescale / YaRN）——呼应结构节



要点：NoPE 负责『能外推』，合成长依赖负责『真的会用长上下文』。

训练稳定性：一组『配套』手段

稳不是单点，是组合

手段	作用 / 为什么
Per-Head Muon	Q/K/V 动量按 head 分块逐头正交化，均衡各头更新尺度
K2 weight clipping	沿用 K2 权重裁剪，抑制爆炸
Quantile Balancing	MoE 负载均衡，避免专家训不充分拖慢 EP
MoonViT-V2 从零训练	不用 SigLIP 初始化：联合训练更稳，视觉评测持平
QAT 从 SFT 起	训练即感知 MXFP4/8 量化，训推一致

- Per-Head Muon：整块正交化会让梯度大的头主导共享更新方向 → 逐头让各头更新尺度对齐
- MoonViT-V2 结论很有信息量：规模够大时，对比学习预训练不是多模态 LLM 的必要初始化

要点：每一项都在回答『大规模下怎么不炸』。

04

预训练 Infrastructure（重点）

让 $2.8T \times 1M$ 训得起、训得稳、训得快 —— 算法与系统的协同设计

总览：并行组合与三大痛点

四层并行 + 三个待解痛点

3T 级预训练并行组合

PP (interleaved 1F1B + VPP)	93 层按 stage 切 · VPP 压 bubble
EP (Expert Parallelism)	896 专家按 rank 切 · all-to-all dispatch/combine
ZeRO-1 DP	优化器状态分片 · 扩批量
Pipeline ZeRO-2 梯度分片	梯度按 DP 分片 + 下沉 CPU
CP (KDA Context Parallelism)	1M 序列切分 · KDA 递归态用 KCP

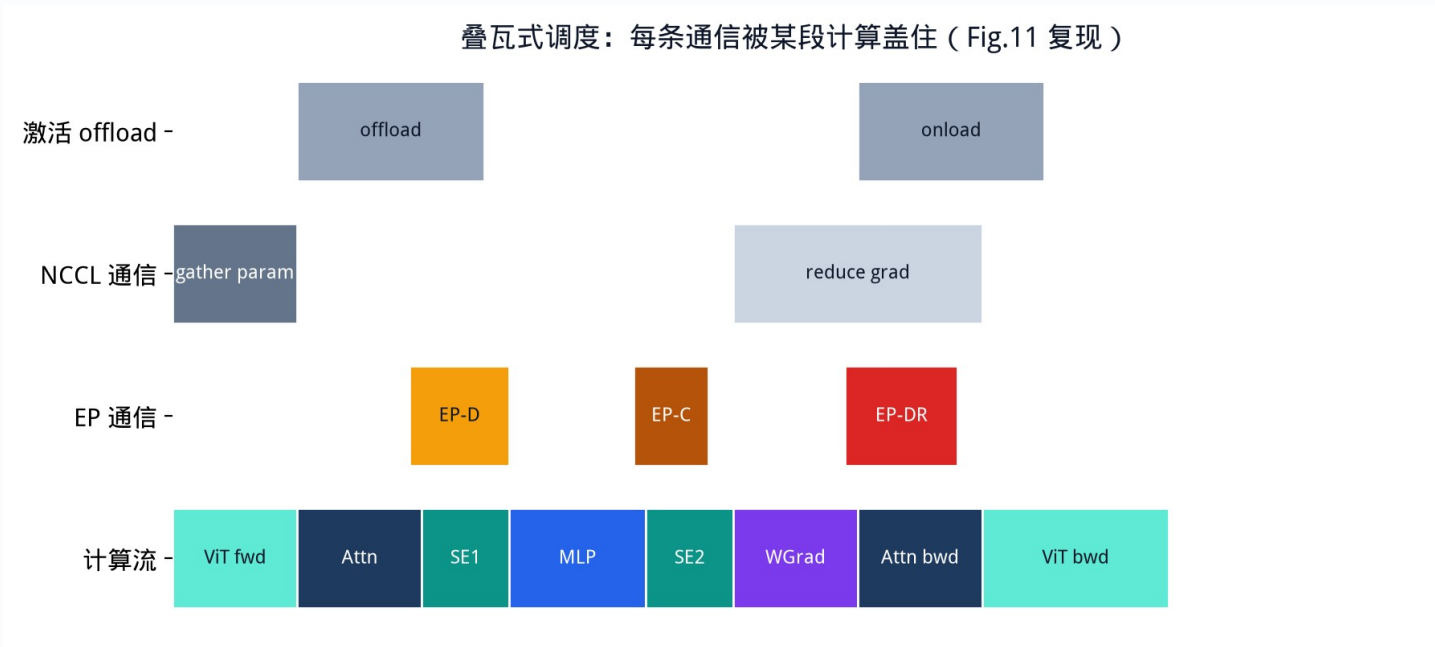
关键约束：EP 的 all-to-all / PP 的 P2P / DP 的 reduce 争同一带宽 → 用计算盖住每一条通信

- PP + VPP : 93 层切流水段, interleaved 1F1B 减 bubble
- EP : 896 专家分片到各 rank , all-to-all 做 dispatch/combine
- CP (KCP) : 1M 序列切段, 固定大小 all-gather 同步递归状态 (KDA 特有)
- ZeRO-1 DP + Pipeline ZeRO-2 : 优化器状态 / 梯度分片, 梯度可 offload CPU
- 三大痛点 = 本节解题顺序: ① EP 负载不均 ② 显存超预算 ③ ViT 计算方差

要点：这套组合的精髓：切完之后每条通信都被某段计算盖住。

叠瓦式调度：每条通信都被计算盖住

Fig.11 调度复现

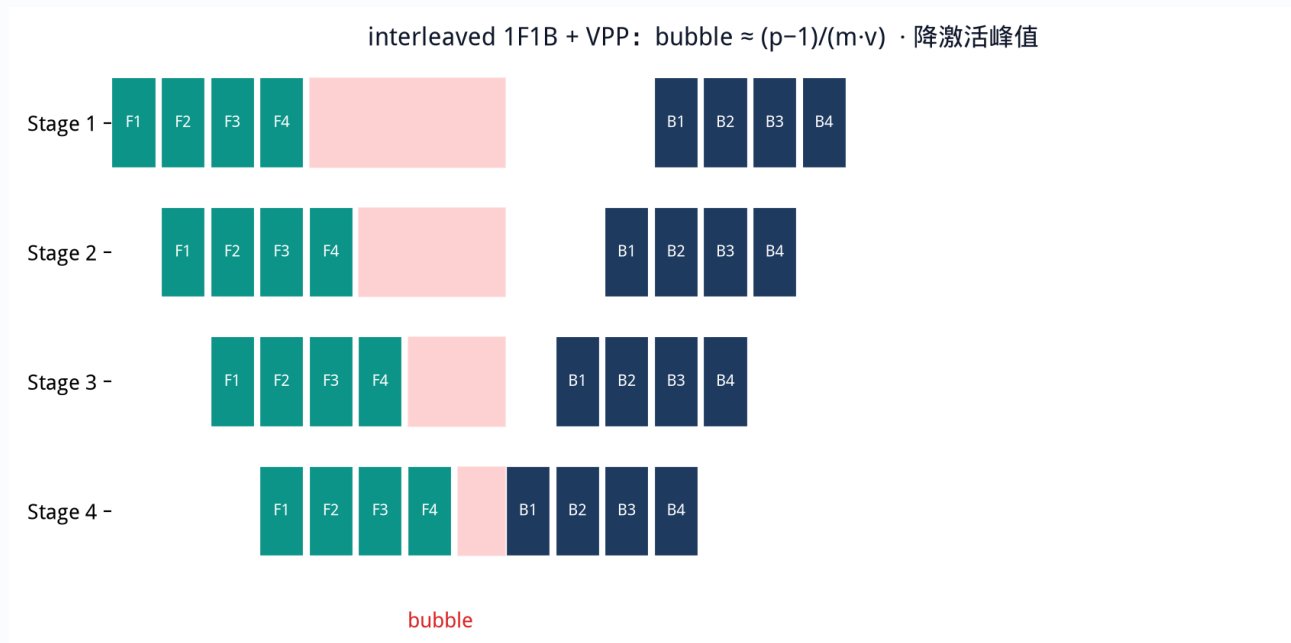


- shared expert 拆 SE1/SE2 派独立 stream → 填 all-to-all 空档
- reduce grad = reduce_scatter + onload + add + offload （ Pipeline ZeRO-2 + CPU 梯度 ）
- EP-DR （ dispatch 重算 ）在反向 → memory-efficient MoE

要点：通信藏进计算的缝隙——这是后续所有优化能成立的基础。

interleaved 1F1B + VPP : 压 bubble , 留一个要治的尾巴

$$\text{bubble} \approx (p-1)/(m \cdot v)$$



- 1F1B : 不改 bubble 比例, 但每 stage 只驻留约 p 份在飞激活 → 降峰值
- VPP : 每物理 stage 再拆 v 个虚拟块 → bubble 再除以 v , 代价是通信 $\times v$
- 副作用: 暖机期各 stage 驻留激活不均 → 后面用『PP 激活再平衡』治

要点: VPP 把 bubble 再除以 v , 代价是暖机期激活不均 (后面会治)。

KDA 算子①：FlashKDA —— 把慢路径榨干

双 kernel 流水 + CHUNK=16 的三重动机

FlashKDA：双 kernel 流水，把递归的低并行度藏起来

朴素单 kernel



FlashKDA 双 kernel 流水



CHUNK = 16 的三重动机

- ① 配合 lower-bounded decay: 16-token 累积衰减落进 BF16, 免复杂 rescale
- ② 16×16 求逆用 Neumann 级数直接展开很便宜
- ③ 可映射到 SM80 的 MMA 指令 -> 多种 NVIDIA GPU 可移植
片上状态 bf16 省一半 shared memory

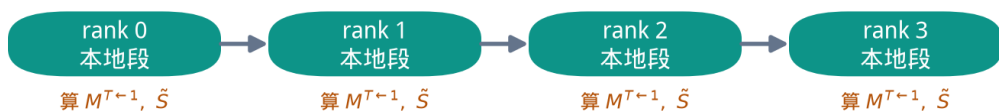
- 问题：chunkwise 里 token 并行阶段被低并行度递归卡住，SM 大量闲置
- 双 kernel 流水：K1（token 并行：门 / 归一 / 衰减 / 求逆）+ K2（head 并行：跨 chunk 递归）
 - 拆开后两阶段独立调优 + 重叠 -> 端到端 ≥ 15% 提速
- **CHUNK=16**：数值落进 BF16（配 lower-bounded decay）/ 16×16 逆用 Neumann / 映射 SM80 MMA 可移植

要点：呼应结构节：lower-bounded decay 让 CHUNK=16 的数值范围成立。

KDA 算子②：KCP —— 线性注意力的上下文并行

转移矩阵分解 + 前缀扫描

KDA Context Parallelism: 转移矩阵分解 + 前缀扫描



一次固定大小 all-gather 交换 (M, S) → 前缀扫描恢复各 rank 入态

线性注意力只传固定大小状态 (softmax 需传随长度增长的 KV 块)

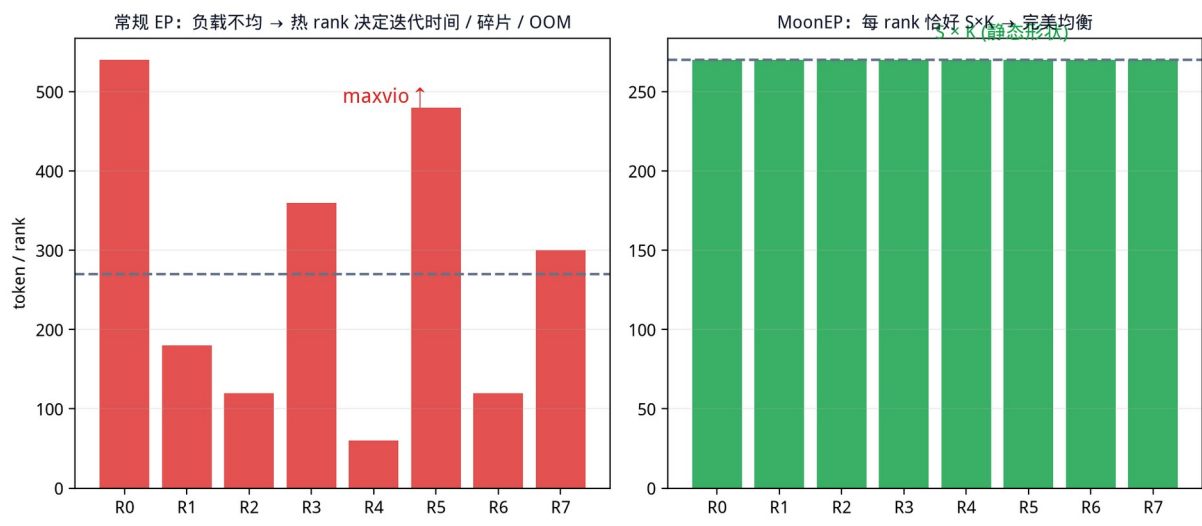
vanilla 「从 S=0 求和」对 KDA 不成立: 转移矩阵 M_t 要作用在入态上

- softmax 的 CP (Ring Attention) 要交换随序列增长的 KV 块; 线性注意力只需传固定大小状态
- 但 delta rule 不能简单求和: 转移矩阵 M_t 作用在入段状态上
- KCP: 每段分解成『累积转移 M 』+『零初值局部状态 $\tilde{S}$ 』两个本地可算量
- → 一次固定大小 all-gather + 前缀扫描精确重组; 通信与序列长度无关 → 线性扩展

要点: 思想迁移: softmax 的 CP, 用转移矩阵分解 + 前缀扫描搬到线性注意力。

MoonEP①：让每个 rank 恰好算 $S \times K$ 个 token

常规 EP 三痛，其实同源



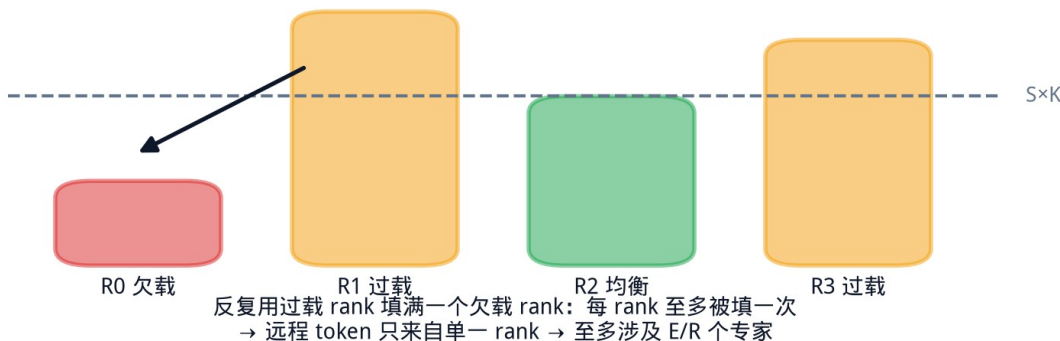
- 常规 EP 三痛：热 rank 决定迭代时间；动态形状→显存碎片直至 OOM；每层 host-device 同步取形状
- 三点同源：都是『token→专家映射动态且不均』的后果
- MoonEP 用动态冗余专家做到完美均衡：热专家临时复制到别的 rank、分流 token
 - 前向在线规划冗余专家并预取权重；反向把冗余梯度 reduce 回 home rank

要点：不去缓解不均，而是直接消灭它——后面所有红利从这来。

MoonEP②：为什么『完美均衡』是可证明的

E/R 冗余专家上界，区别于 ECHO/UltraEP 的根本

定理：每 rank $\leq E/R$ 个冗余专家的均衡方案必然存在（界基本紧）



预留 E/R 冗余槽 ⇒ 规划永远可行、训练永不中断（对比 ECHO/UltraEP 会停训）

- 定理：任意路由下，每 rank 只需 $\leq E/R$ 个冗余专家就一定能均衡（ E = 专家数， R =EP 并行度）
 - 证明：反复用超载 rank 填满欠载 rank，每 rank 至多被填一次 → 远端 token 只来自单一 rank → 至多 E/R 个专家
- 下界 $\approx E/R \rightarrow$ 界基本紧；预留 E/R 槽位后规划永远可行、训练永不中断
 - 对比 ECHO/UltraEP：固定冗余数或 token cap，无解时被迫停训 + 手调

要点：把『确定性』当一等目标：永不中断、静态形状、零拷贝。

MoonEP③：完美均衡换来的两个红利

零拷贝 + 静态形状

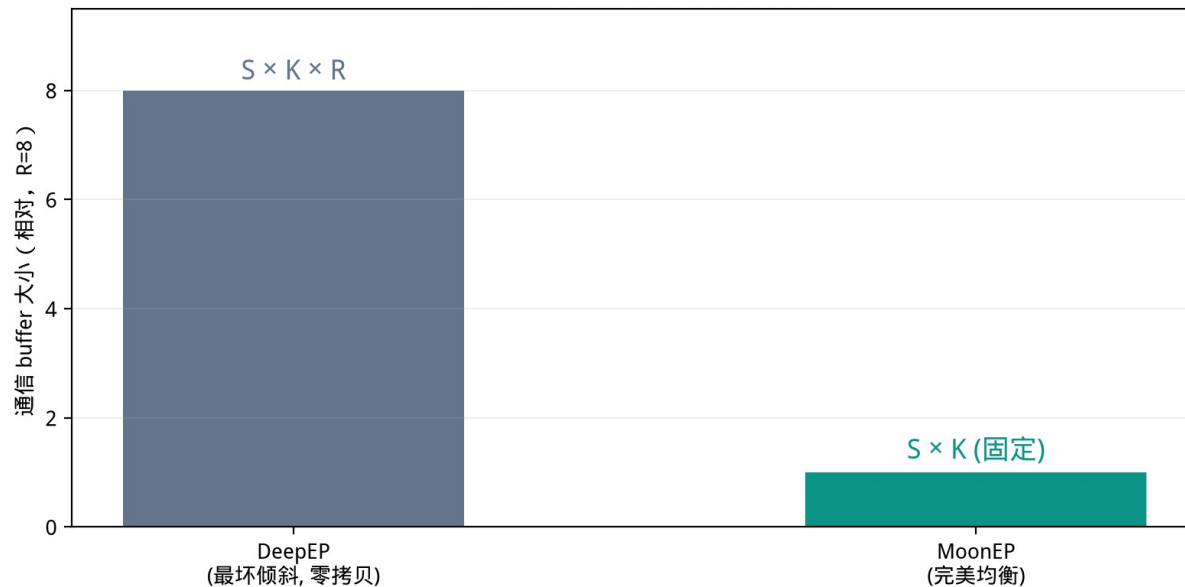
- **零拷贝**：规划 kernel 预算每 token 目的地 → 直接写到远端最终槽位，buffer 视图交给计算

- DeepEP 最坏要 $S \times K \times R$ 的 buffer；MoonEP 完美均衡恒定 $S \times K$ ，与倾斜无关

- **静态形状**：每层 MoE 形状静态已知 → 消除 host 同步、降 kernel 启动开销

- rank 内仍偏斜 → workload-aware GEMM 调度（代价模型 + 离线 autotune）+ 共享专家单独 stream

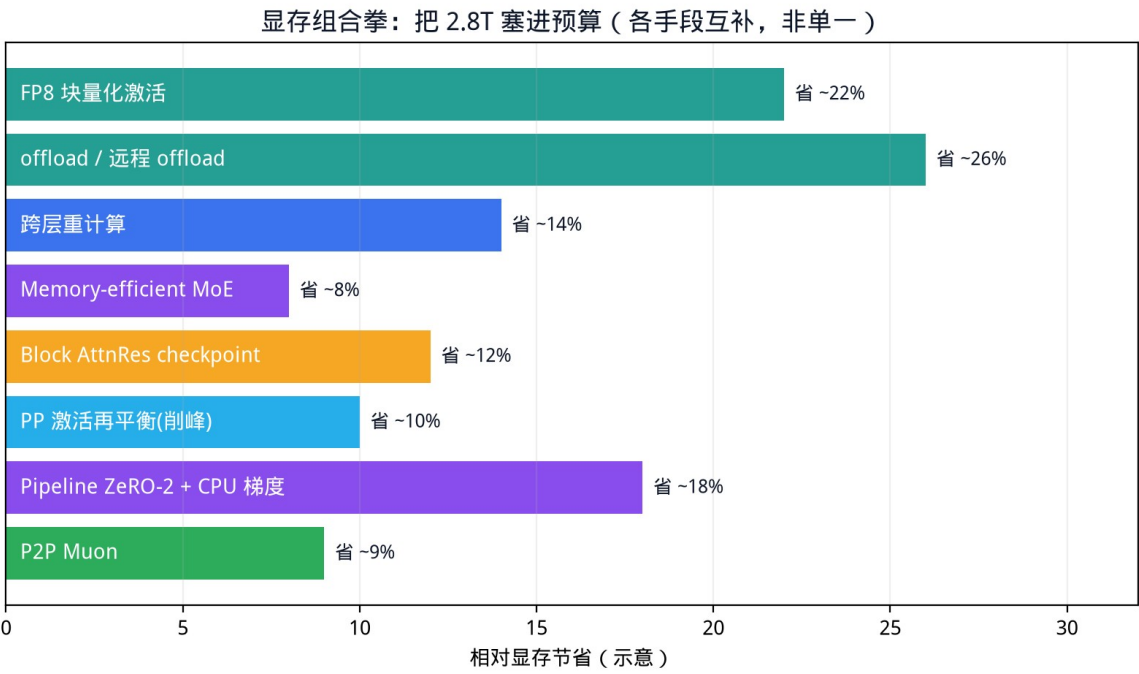
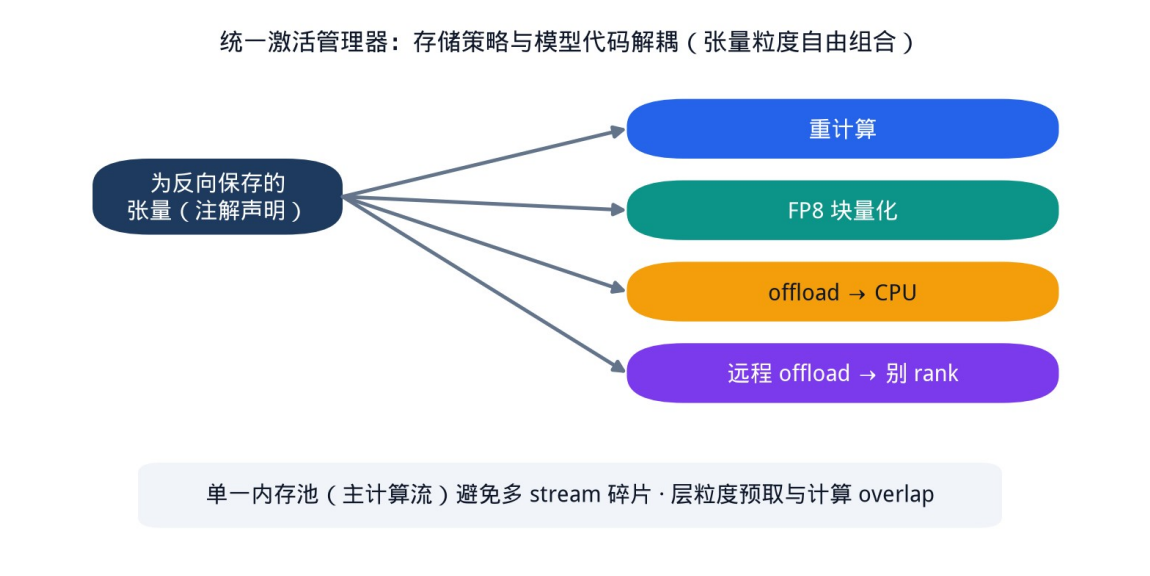
零拷贝的代价：buffer 大小对比



要点：完美均衡是因，零拷贝 / 静态形状 / 无碎片都是果。

显存工程：把『省显存』做成可组合的存储策略

统一激活管理器 + 账本



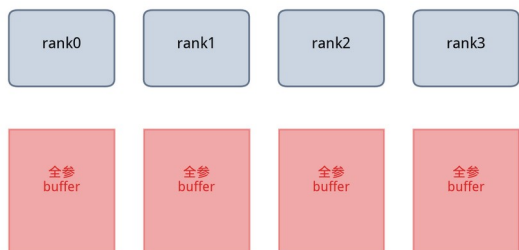
- 每个为反向保存的张量挂『可插拔存储后端』：重计算 / FP8 块量化 / offload / 远程 offload，张量粒度自由组合、单内存池避免碎片
- MoE：梯度数学变换免存前向输出 + 反向重算 dispatch（通信与反向计算重叠）；Block AttnRes：整体 checkpoint → 显存足迹 = 标准残差

要点：没有银弹——多种手段各省一点，合起来才把 2.8T 塞进预算。

显存工程（续）：三个系统级手段

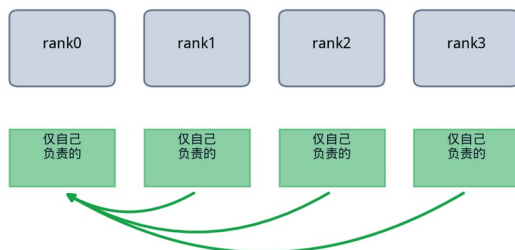
把压力从 HBM 挪到 CPU / 别的卡 / P2P

朴素：全量 all-gather



显存 $O(P)/rank$ + all-gather $O(P)$ 通信

K3：P2P 定向拉分片



buffer $O(P/DP)$ · 消除全参 buffer · 通信流水隐藏

- PP 激活再平衡：1F1B 暖机各 stage 激活不均 → Mooncake 远程 offload 到别的 PP rank 削峰

- Pipeline ZeRO-2 + CPU 梯度：梯度分片下沉 CPU，GPU 只留 double buffer

- P2P Muon：正交化需完整矩阵；改为只 P2P 拉『自己负责更新的』分片

- 消除全参 all-gather 的显存 + 通信双爆炸，buffer 从 $O(P)$ 降到 $O(P/DP)$ ，按 model-chunk 流水隐藏

要点：削峰而非降总量——显存瓶颈是『单卡峰值』，不是『总量』。

多模态编码器：把 ViT 藏进流水线气泡

动态 CP + bubble filling

Bubble filling: 把 ViT 计算塞进 PP 流水气泡 (承 K2.5 DEP · 思路同 Optimus)



ViT (可搬动、无跨 micro-batch 依赖) → 有效开销基本被隐藏

- 痛点③：大图 / 长视频让 ViT 计算量方差大，放关键路径会拖慢流水、造成设备间负载不均
- 动态 CP：单张大图沿 patch 维切到多设备，跨 CP rank gather-KV 算注意力
 - CP 组再分 sub-group，多张大图负载均衡分散，通信占比不随规模膨胀
- 塞进 PP 气泡：首批 micro-batch 的 ViT 前向同步先跑，其余 ViT 前 / 反向调度进气泡
 - → ViT 有效开销基本被隐藏 (承 K2.5 DEP，思路同 Optimus)

要点：原生多模态里视觉不是附加项，要和文本在系统层深度融合。

对照： K3 vs DeepSeek 的训练 Infra 气质

两种工程审美

主题	DeepSeek (V3/V4 公开)	Kimi K3
EP 通信	DeepEP : 高效 all-to-all , 容忍不均	MoonEP : 动态冗余做到完美均衡 + 静态形状
负载均衡	EPLB (部署期) + sign 偏置	训练期每步在线规划 + Quantile Balancing
低精度	FP8 训练 (块量化)	FP8 存激活 + 后训练 MXFP4/8 QAT
流水线	DualPipe 双向流水降 bubble	1F1B+VPP , 把 ViT/offload/EP 通信塞进 bubble
长上下文	NSA/DSA/CSA 稀疏 kernel	FlashKDA + KCP (线性注意力专属)

- 共同点：架构为系统而生 (MLA/KDA 都是为 cache 与通信设计的注意力)
- 差异： K3 把训练期确定性当一等目标； DeepSeek 更极致做通信 - 流水线重叠

要点：没有绝对优劣——确定性 vs 极致重叠，两种审美。

05

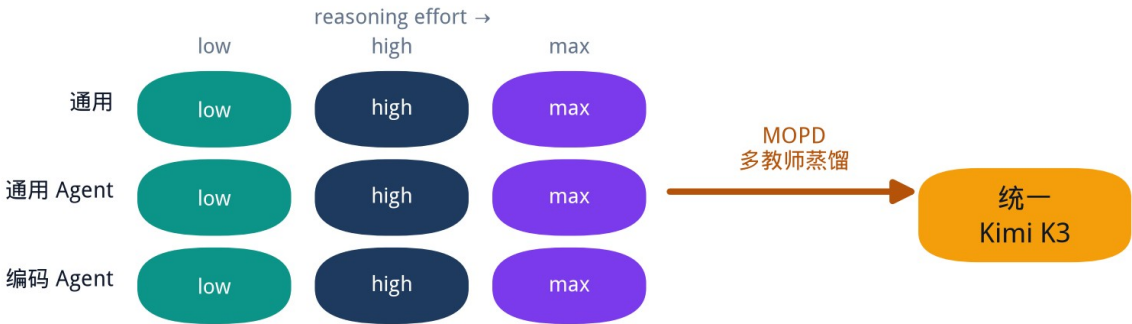
强化学习（简化）

SFT → 分域分强度 RL → 多教师蒸馏；重点看『范式』而非细节

后训练三阶段：分而治之，再合并

9 个专家 → 一个统一模型

后训练：分域 × 分强度练 9 个专家 → 多教师蒸馏合并



RL FLOPs ↑ → 工具调用步数与综合能力同步增长（长程自主执行是学出来的）

- SFT：合成复杂 agentic 轨迹 + XTMl 模板统一序列化；QAT 从此开始（MXFP4/8）
- RL：3 域 × 3 reasoning effort = 9 个专家
 - RL FLOPs ↑ → 工具调用步数与能力同步增长（长程自主执行是学出来的）
- MOPD：多教师 on-policy 蒸馏合并 —— 与 DeepSeek-V4 / MiMo-V2 范式收敛

要点：不给每个任务单独训，而是『域 × 强度』九宫格再蒸回一个。

RL 的关键设计与环境（快速过）

四个关键设计 + 六类环境

统一白盒 harness	知识图谱任务合成	Kernel 优化 (+ 反 hacking)
个人助理 mock 应用	AET 自主执行	WebDev 多 scaffold

- Partial rollout：完成 λ 比例即优化，未完轨迹入队下轮续跑 → 长尾不阻塞；per-token 正则容忍极端 off-policy
- Reasoning-effort 预算：超 $\tau \cdot b_o(x)$ 的轨迹 reward=-1； τ 退火得 max/high/low
- Agentic GRM（不可验证任务）：锦标赛两两比较 + rubric 协议 + 长度阈值防刷分
- 训练 + 评测全程创建 5,100 万 + 沙箱 / 150 万 + 镜像（AgentENV microVM）——环境工程 = 核心壁垒

要点：环境工程（沙箱 /harness/ 任务合成）是新的核心壁垒。

蒸馏合并、量化与投机解码

让训练目标对齐部署现实

- **MOPD**：学生在自己分布上采样、教师逐 token 打分 (dense reward) → 避免 SFT 蒸馏的 exposure bias
 - reward = 教师 / 学生对数概率比, clip 到 $\pm R_{\max}$; 无缝接入 RL 框架、复用 partial rollout
- **QAT 贯穿 SFT+RL**：MoE 权重 MXFP4 / 激活 MXFP8 , 非专家部分高精度; rollout 与训练同量化 → 训推一致
- **投机解码**：复用预训练 MTP 层做 EAGLE-3 draft ; LK loss 直接优化接受率而非 KL 代理
- **贯穿主线**：QAT 对齐量化 · LK loss 对齐接受率 · on-policy 对齐推理分布

要点：训练目标尽量对齐部署现实——这是很多 RL 翻车的隐形来源。

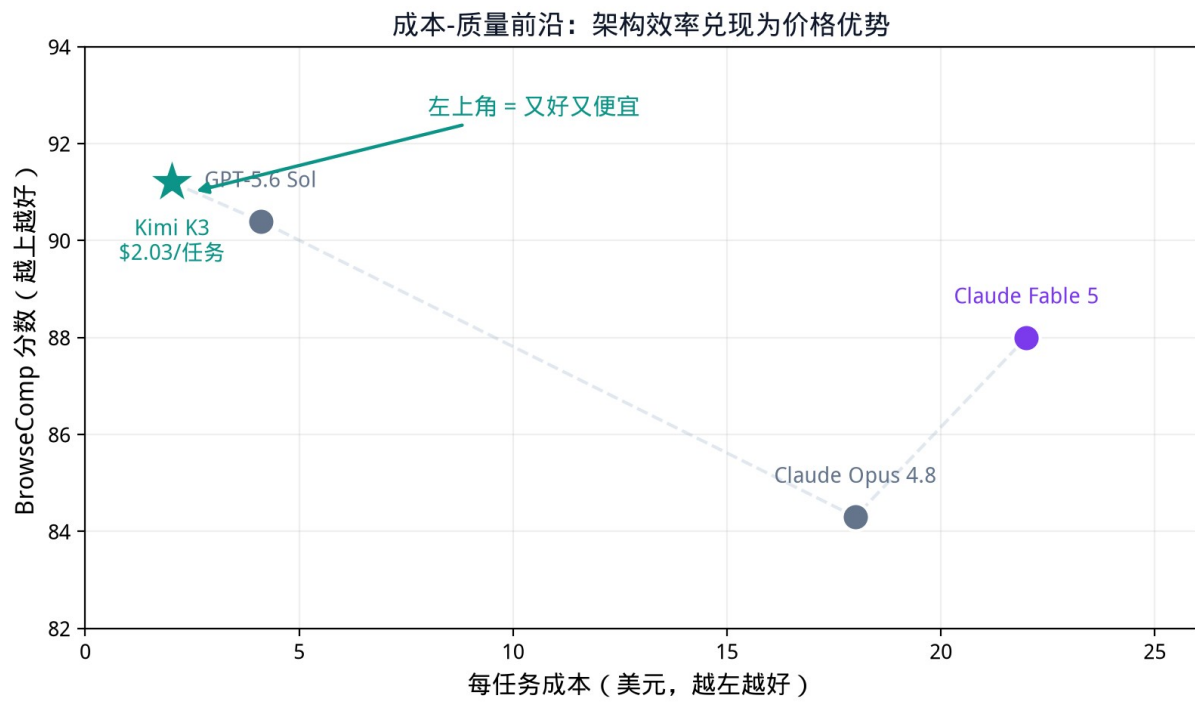
06

评测、成本与总结

开源新前沿的证据，以及可以带走的方法论

评测与成本效率

开源第一梯队 + 成本 - 质量前沿

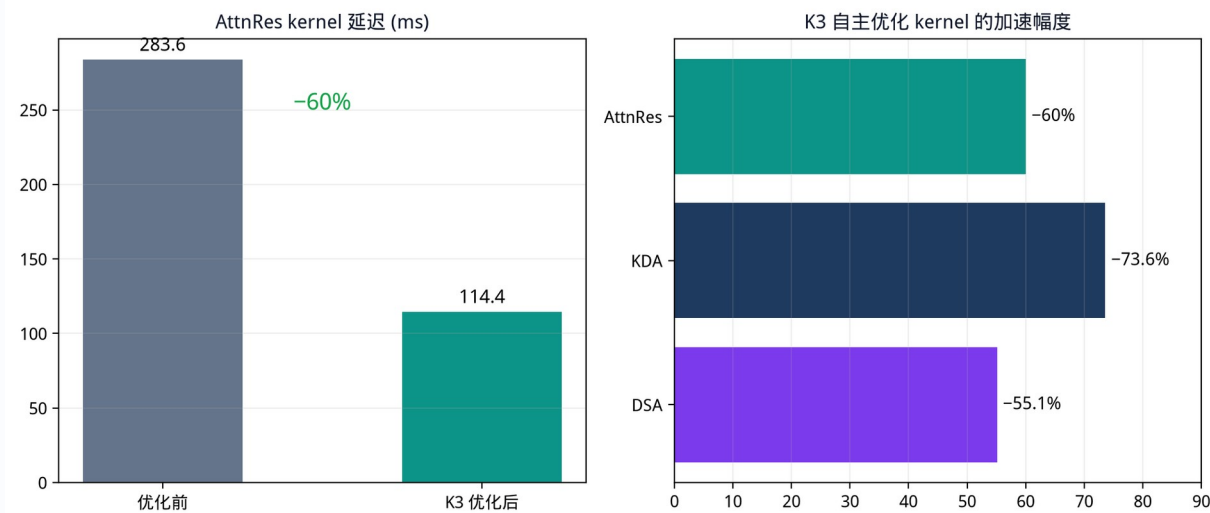


- 整体：Fable 5 $\geq$ GPT-5.6 Sol > Kimi K3 > Opus 4.8 / GPT-5.5 / GLM-5.2；研究级推理仍有差距
- 亮点：BrowseComp 91.2（第一）· ProgramBench 77.8（第一）· SWE-Marathon 42.0（领先 Fable 5 七点）
- 第三方：AA 智能指数 #4/580；WebDev Arena 开源首次登顶（Elo 1678）
- 成本：BrowseComp 最高分且仅 \$2.03/任务 = GPT-5.6 Sol 的一半、Claude max 档的十分之一量级

要点：架构效率最终兑现为价格优势——呼应开场。

案例：模型开始反哺自己的 Infra

能力 -Infra 飞轮



- GPU kernel 优化 (24h/ 任务) : AttnRes 283.6→114.4ms ; KDA -73.6% ; 追平 Fable 5

- 早期 K3 checkpoint 已承担团队内多数 kernel 优化工作

- MiniTriton : 自研 DSL→MLIR→PTX 编译器; L20 上 matmul 达 cuBLAS ~90%

- nano-kpu : 48h 自主跑通开源 EDA , 4mm²/100MHz/8700 tok·s⁻¹ 芯片原型

- 安全: Linux kernel 发现 16 个未知漏洞 (含远程堆越界写、Dirty-COW 类提权)

要点：用 RL 环境练 kernel → 模型替团队写 kernel → 支撑下一代训练。

总结：可以带走的五条

一页收束全场

#	带走	一句话
1	为什么这样设计	KDA/AttnRes/LatentMoE 各解一维信息流瓶颈；混合注意力解耦『成本 vs 检索』
2	算法 - 系统协同	lower-bounded decay 换 Tensor Core、NoPE 免调参、QAT 对齐量化——每层为下一层让路
3	确定性即性能	MoonEP 完美均衡 + 静态形状 + 永不中断；把随机性与长尾挤出训练
4	方法论可迁移	比较技术前先各自调最优（cosine vs WSD）；存储策略抽象；QB 直方图均衡
5	能力 -Infra 飞轮	模型优化 kernel → 支撑下一代训练；环境 / 沙箱工程成核心壁垒

- 一句话：K3 用 2.8T 证明了『线性混合路线』能撑起 frontier 模型 + 1M 上下文 + 可负担的服务成本

要点：最好的系统优化，往往始于一个小的算法改动。

Q & A · 谢谢！

常见问题预案：KDA vs Mamba/GLA（delta rule 纠错 + 通道级门）· 为何不用纯稀疏 attention · MoonEP 与 DeepEP/EPLB 差异 · NoPE 外推 · 896 选 16 如何训得起

细节与公式详解见 [DESIGN_NOTES.md](#) / [PRETRAIN_INFRA_DEEPDIVE.md](#)